Fondamenti di programmazione

Canale A-L - Prof. M. Mancini

AA 26-27

Lezione 3

RECAP lezione 2

- stringhe, stringhe multilinea, interpolazione
- variabili, riferimenti e namespace
- assegnamento
- conversione stringhe <-> numeri
- confronti

Operatori di assegnamento 'potenziato' precedendo = con l'operatore

Può far comodo modificare una variabile aggiungendole, sottraendole, moltiplicandola o dividendola per un valore

- `A += 1` **incremento** (come `A = A+1` ma più compatto)
- `A -= 2` **decremento** (come `A = A-2` ma più compatto)
- `A *= 3` **moltiplicazione** (come `A = A*3` ma più compatto)
- `A /= 4` **divisione** (come `A = A/4` ma più compatto)

In [ ]:

```
# Esempio
A = 42
print(A)
A += 1
print('più    1', A)
A -= 12
print('meno  12', A)
A *= 10
print('per   10',A)
A //= 5
print('diviso 5',A)
A %= 7
print('il resto per 7',A)
A **= 2
print('elevato alla 2',A)
B ='Paperino '
B += 'e Minnie'
print(B)
```

In [ ]:

```
# Se applicati alle stringhe, gli assegnamenti potenziati '+=' e '*='
# eseguono concatenazioni o ripetizioni
A = 'Paperoga '
print(A)
A += 'e Gastone | '
print(A)
A *= 3
print(A)
```

Assegnamenti multipli

In Python possiamo assegnare contemporaneamente più variabili in una sola istruzione

```
A, B, C = 1, 2, 3

A contiene 1
B contiene 2
C contiene 3

Basta che il numero di espressioni calcolate a destra sia uguale al numero di variabili a sinistra

A destra può esserci un qualsiasi contenitore list/tuple/dict/set (vedi dopo)
```

In [ ]:

```
# Esempio
L = 'Paperino e Minnie'.split()    # produce la lista ['Paperino', 'e', 'Minnie']
P, e, M = L                        # assegno i 3 elementi della lista a 3 variabili
print(P)                           # e le stampo
print(e)
print(M)
```

Sono ESTREMAMENTE COMODI per

- estrarre informazioni organizzate in una lista o tupla per **posizione**
- scambiare tra loro i valori di due o più variabili
- spacchettare i valori di ritorno di una funzione (lo vediamo dopo)

In [ ]:

```
# ESEMPIO: spacchettiamo una lista che contiene i dati di una persona, organizzati per posizione
#          nome      cognome    età altezza figli
SCHEDA = ['Andrea', 'Sterbini', 64, 186,    2]
nome, cognome, eta, altezza, figli = SCHEDA
print('nome\t',nome)
print('cognome\t',cognome)
print('età\t',eta)
print('altezza\t',altezza)
print('figli\t',figli)
```

In [ ]:

```
# ESEMPIO: scambiamo due variabili
prima = 'Paperino'
seconda = 'Minnie'
prima, seconda = seconda, prima  # scambio i valori
print(prima)
print(seconda)
```

Come mai lo scambio funziona?

Se corrispondesse alla sequenza di istruzioni

```
prima = seconda
seconda = prima
```

otterremmo che in entrambe le variabili c'è 'Minnie'

In [ ]:

```
prima = 'Paperino'
seconda = 'Minnie'
prima = seconda
seconda = prima
prima, seconda
```

Invece quello che fa Python è:

- PRIMA calcola l'espressione a destra della `'='` ovvero **`seconda, prima`** che produce la coppia **`('Minnie', 'Paperino')`**
- poi assegna a ciascuna delle due variabili a sinistra dell' `'='` i due valori della coppia

Dato che i due valori sono stati entrambi già letti dalle variabili, non si perdono e possono essere assegnati senza problemi

In [ ]:

```
# Esempio contorto
A, B, C = ['uno', 'due', 'tre']

B, A, C, C, B, A = A, A, A, A, C, B       # solo gli ultimi 3 assegnamenti hanno significato :-D
A, B, C                                   # non ci siamo persi nulla!
```

Condizioni e percorsi alternativi

Per scegliere di eseguire parti diverse di codice in situazioni diverse si usa l'istruzione **`if`** e delle condizioni (booleane) che valgono **True** o **False**

Sintassi:

```
if condizione1:
    istruzioni eseguite se condizione1 è True
elif condizione2:          # sezione opzionale (e ripetibile + volte)
    istruzioni eseguite
    se condizione1 è False e condizione2 è True
else:                      # sezione opzionale
    istruzioni eseguite
    se sia condizione1 che condizione2 sono False
```

NOTATE CHE: il codice è organizzato in modo particolare, i blocchi di istruzioni sono **indentati** (spostati a destra) rispetto alla parola chiave che li introduce (if, elif, else)

Indentazione .... ma come mai?

In C/C++/Java usiamo le parentesi graffe **{ }** per raggruppare i blocchi di codice e i punto-e-virgola **;** per separare le istruzioni
Questo rende possibile scrivere codice molto disordinato e poco leggibile come questo su una sola riga

```
if(condizione1){istruzione1;istruzione2;istruzione3;}else{istruzione4;istruzione5;istruzione6;}
```

Negli anni si è diffuso uno stile di scrittura ordinato e più leggibile, come questo:

```
if (condizione1) {  // con le strutture di controllo ben visibili
    istruzione1;    // con una istruzione per ciascuna riga
    istruzione2;    // commentata se serve, sulla stessa riga
} else {
    istruzione3;    // e ciascun blocco logico di istruzioni
    istruzione4;    // spostato un po' a destra per evidenziarlo
}
```

Questo stile ***indentato*** è molto leggibile e fa parte delle regole obbligatorie di scrittura del Python
E visto che ogni istruzione è su una riga diversa e i blocchi sono indentati, non c'è più bisogno di parentesi graffe e punto-e-virgola

Altrimenti possiamo usare l'istruzione `match` (simile allo switch/case di C/C++/Java)

Serve a confrontare un valore con più valori possibili, **esaminandone anche la struttura!**

```
match espressione:
    case valore1:              # match esatto
        istruzioni se variabile == valore1
    case valore2 | valore3:    # match con più alternative
        istruzioni se variabile == valore2 OPPURE valore3
    case [x, y]:               # match strutturale
        istruzioni se variabile è una lista di 2 valori qualsiasi
        i valori x ed y vengono spacchettati e possono essere usati in questo blocco
    case _:                    # caso di default
        istruzioni se variabile non è uguale a nessuno dei valori precedenti
```

NOTA: diversamente da C/C++/Java, non c'è bisogno di `break` alla fine di ciascun blocco perchè i diversi blocchi sono mutualmente esclusivi (ne viene eseguito solo uno oppure nessuno)

Cosa stampa il programma ?

In [ ]:

```
# soluzione
is_raining, got_umbrella, have_money   = True, False, True
if not is_raining:                                          # False
    print('go out')
elif not got_umbrella and have_money:                       # True
    print('buy umbrella and go out')
elif got_umbrella:                                          # non viene esaminato
    print('go out since got umbrella')
else:                                                       # non eseguito
    print('stay home')
```

In [ ]:

```
# soluzione
X = "Paperino ama Paperina".split()
match X:
    case 42:
         print("La risposta alla domanda fondamentale sulla vita, l'universo e tutto quanto")
    case "Paperino ama Paperina":
         print("Ma anche Gastone?")
    case "Paperino" | "ama" | "Paperina":
         print("E Topolino ama Minnie")
    case [ soggetto, verbo, oggetto ]:
         print( f"{oggetto} è {verbo}to/a da {soggetto}" )
    case _:
         print( "Non ho capito cosa hai detto" )
```

Cicli ed iterazione/ripetizione

Caso 1: quando *SAPETE QUANTE* iterazioni/ripetizioni dovete fare

ALLORA è meglio usare il FOR

Sintassi:

```
# viene definita la variabile 'elemento' e gli viene
# via via assegnato il prox valore della sequenza
for elemento in sequenza:
    blocco di codice da ripetere
    per ciascun elemento della sequenza
else:        # opzionale
    blocco che viene eseguito se si è completato il
    ciclo normalmente senza interruzioni (break) o errori
istruzioni dopo il for
```

In [ ]:

```
# ESEMPIO
for X in [1, 2, 3, 4]:  # ripeto 4 volte, con X che prende i valori 1, 2, 3, 4
    print(X)            # l'istruzione print
```

Nel corpo del ciclo posso usare le istruzioni

```
break    # per uscire immediatamente dal ciclo
         # e proseguire con le istruzioni DOPO
         # tutto il blocco FOR
continue # per saltare al prossimo elemento della
         # sequenza senza completare le altre istruzioni del blocco indentato
```

In [ ]:

```
# ESEMPIO con break e continue (a seconda del valore)
for X in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
    if X == 13:
        break      # esco se incontro il valore 13
    if X % 2 == 0:
        continue   # salto tutti i numeri pari
    print(X)
else:
    print('li ho stampati tutti') # solo se ho completato il FOR senza break
```

Come contare: con il generatore di sequenze `range`

E' un generatore di sequenze di interi, cioè un oggetto che fornisce un numero per volta

chiamandolo così	produce uno dopo l'altro la sequenza di valori
`range(fine)`	**0, 1, 2, 3, 4.... (fine-1)**
`range(inizio, fine)`	**inizio, inizio+1, ... , fine-1**
`range(inizio, fine, incremento)`	**inizio, inizio+incremento, ..., fine-1**

In Python 3 ci sono moltissimi generatori, che riducono l'uso della memoria

NOTATE CHE: il valore di `fine` non viene mai prodotto, e che range si comporta proprio come una slice

In [ ]:

```
## range crea un oggetto che fornisce un nuovo valore ogni volta
## che gliene viene richiesto uno nuovo dal 'for' o da 'list'
R = range(-12,-30,-3)
print(R, type(R) )      # se lo stampo mi mostra che è un oggetto di tipo 'range'
for i in R:             # se lo uso nel FOR mi genera i numeri 1...9
    print(i, end=' ')
list(R)                 # e posso anche usarlo per costruire una lista di valori
```

Caso 2: quando NON SAPETE QUANTE iterazioni/ripetizioni fare

ALLORA è meglio usare WHILE

```
prima dobbiamo inizializzare le variabili usate nella condizione
while condizione_vera:
    # blocco eseguito SOLO SE LA CONDIZIONE E' VERA
    blocco di codice da ripetere
    # FONDAMENTALE!!!! DA NON DIMENTICARE!!!
    aggiornate SEMPRE i dati della condizione
    altrimenti il ciclo NON SI FERMA PIù
    perchè la condizione resta sempre vera
else:      # parte opzionale (come per il for)
    blocco di codice eseguito solo se si termina il ciclo
    perchè la condizione è diventata False
    ovvero NON siamo usciti con break
codice a seguire
```

In [ ]:

```
# ESEMPIO
X = 0                   # all'inizio X vale 0
while X < 20:           # ripeto ma solo se X è minore di 20
    X += 1              # aggiorno la condizione incrementando X (conviene farlo subito)
    if X == 22:         # quando arrivo a 12
        break           # esco
    if X % 2 == 0:      # altrimenti se X è pari
        continue        # lo salto e passo al prossimo
    print(X, end=' ')   # stampo X (seguito da spazio invece che da accapo)
else:                   # se esco perchè X non è più < 20
    print('\nli ho stampati tutti')       # lo dico
print('\ninizio del codice seguente')   # e poi continuo
```

Quante stampe fa il programma ?

In [ ]:

```
# Soluzione
# Quante volte il programma stampa qualcosa?
for X in range(50, -1, -7):                    # 50, 43, 36, 29, 22, 15, 8, 1
    if X % 5 == 0:                             # salto 50, 15
        continue
    if X // 7 < 5:                             # quando X < 35=7*5 esco
        break
    print(X)                                   # stampo solo 43 e 36
else:
    print("Ho finito")                         # non lo stampo perchè sono uscito con break
print("e ora basta")                           # questo lo stampo, il for è finito
```

In [ ]:

```
# Soluzione
# Quante volte il programma stampa qualcosa?
N = 0
while 2**N < 5000:                          # fino a N==12 compreso 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 409
    if N % 3 == 0:                          # salto i multipli di 3 (0, 3, 6, 9, 12) (1 8 64 512 4096)
        N += 2                              # e il valore seguente  (1, 4, 7, 10)
        continue
    if N > 12:                              # questo non succede mai
        break
    print(2**N, N)                          # stampo potenza e indice
    N += 1                                  # aggiorno N
else:
    print("Ho finito")                      # stampo, NON sono uscito col break
print("facciamo un intervallo?")            # il while è finito, si stampa
```

FINO QUI

Modello dei dati di Python

DUCK typing: ”se cammina come una papera e fa ’quack’ come una papera, allora è una papera”

Nel *modello dei dati* del Python vogliamo che due oggetti siano **intercambiabili** quando 'si comportano allo stesso modo'

'si comportano allo stesso modo' == 'hanno gli stessi metodi'
(NOTA: metodi con nomi uguali potrebbero avere realizzazioni interne diverse)
i numeri interi e i float sono un esempio (e anche i razionali e i complessi)
E' quindi possibile costruire nuovi tipi di oggetti che si comportano in modo simile ad altri
(vedremo in seguito come farlo)
NOTA: questo assomiglia alle Interface in Java

NOTA: incapsulamento delle informazioni

gli attributi (informazioni interne) degli oggetti in Python NON sono protetti da sguardi indiscreti ed è possibile leggerli o modificarli
In altri linguaggi compilati (Java, C++) **è possibile 'nascondere'** le informazioni interne dell'oggetto e fare in modo che solo i metodi dell'oggetto le possano manipolare
In Python (interpretato) questo non è possibile (o è stato scelto così)

Ma si è stabilito che per convenzione che:
TUTTI GLI ATTRIBUTI E METODI CHE INIZIANO CON '_' (underscore) vanno considerati PRIVATI
e **non vanno letti o modificati o usati direttamente**
(chi li ha inventati potrebbe modificarli in future versioni del programma)
quindi **NON USATE MAI DIRETTAMENTE** i metodi e gli attributi che iniziano per '_'
(vedremo poi come definirli)

Esistono alcuni **metodi speciali** (che iniziano per `__` doppio underscore) che realizzano le funzionalità degli operatori usati nelle espressioni e di alcune istruzioni. Per es.

- `__eq__` che implementa l'operatore di confronto `==`
- `__add__` che implementa l'operatore di somma `+`
- `__mul__` che implementa l'operatore di prodotto `*`
- `__len__` che implementa la funzione `len`

Tutti gli oggetti che implementano questi metodi possono essere confrontati, sommati, moltiplicati, contati, ...

Ecco come mai le stringhe possono essere sommate o moltiplicate, hanno un proprio metodo `__add__` che concatena e `__mul__` che ripete la concatenazione
Questo vi sarà utile quando vedremo come costruire oggetti che possiamo confrontare sommare, o di cui vogliamo calcolare la lunghezza (se contengono elementi) ...
(nelle lezioni sulla matematica dei colori)

In [ ]:

```
N = 3
N.__add__(5)    # questo è cosa succede veramente se scrivo N + 5
```

In [ ]:

```
# i numeri complessi hanno le 4 operazioni come gli altri numeri
# (ma sono implementate diversamente)
X_complesso = 3 + 2j     # si usa 'j' al posto di 'i'=sqrt(-1)
Y_complesso = 7 - 6j
```

In [ ]:

```
# somma complessa: si sommano separatamente le parti reali e quelle immaginarie
Z_complesso = X_complesso + Y_complesso

# vediamone le componenti, che sono negli attributi 'real' e 'imag'
print(Z_complesso, 'reale=', Z_complesso.real, 'immaginaria=', Z_complesso.imag)

# la stessa cosa di fare
X_complesso.__add__(Y_complesso)
```